Système d'exploitation
CM3 *Threads*

Pierre Rousselin
rousselin@univ-paris13.fr

Université Sorbonne Paris Nord
L2 informatique et double-licence

février 2026

Motivation

Pointeurs génériques

Pointeurs de fonctions

Threads

Motivation

Pointeurs génériques

Pointeurs de fonctions

Threads

Threads

- Jusqu'à présent, le partage des tâches s'est fait en utilisant plusieurs *processus*,
- ce qui signifie autant d'espaces mémoire
- et une communication possible, mais assez lourde par exemple *via*
  - des signaux (trop rudimentaire) ;
  - des fichiers, par exemple un *pipe* (voir plus loin), une *socket* (plus tard), mais nécessitent d'établir un protocole (même minimal) pour savoir ce que les processus échangent et à quel le moment ils le font.
- Les *threads* (fils d'exécution, aussi appelés processus léger) autorisent un partage des tâches plus flexible au sein d'un même processus.

Partage des tâches, pourquoi ?

- Loi de *Moore* : le nombre de transistor des microprocesseurs double tous les deux ans...
- ... ce qui mécaniquement améliore leur performance,
- Mais depuis les années 2000, ce n'est plus vrai sur un seul processeur (trop de consommation électrique, donc de dissipation thermique).
- Donc, plutôt que de fournir un processeur toujours plus puissant, les constructeurs augmentent le nombre d'unités de calcul (ou cœur, *core*) du processeur,
- et même, dans la plupart des cas permettent à un cœur physique d'effectuer deux tâches en même temps pour devenir 2 cœur logiques (*SMT*, *simultaneous multithreading*
- Sur Linux, la commande `lscpu` permet de connaître le nombre de cœurs et leurs fréquences.
- Tendance actuelle (2026), hétérogénéité des cœurs (certains cœurs économes en énergie, d'autres plus performants).

Fonctions du jour

```
#include <pthread.h>

int pthread_create(pthread_t *thread, const pthread_attr_t *attr,
                   void *(*start_routine) (void *), void *arg);

int pthread_join(pthread_t thread, void **retval);
```

Fonctions du jour

```
#include <pthread.h>

int pthread_create(pthread_t *thread, const pthread_attr_t *attr,
                   void *(*start_routine) (void *), void *arg);

int pthread_join(pthread_t thread, void **retval);
```

Ça fait peur, hein ?
But de ce cours : comprendre ces fonctions.

Motivation

Pointeurs génériques

Pointeurs de fonctions

Threads

Pourquoi typer les pointeurs ?

Qu'apporte le type du pointeur ?

- le nombre d'octets à sauter dans l'arithmétique des pointeurs ;
- le nombre d'octets à récupérer ou à écrire lorsqu'on déréférence ;
- et surtout une aide du compilateur... (qui repère nos erreurs de type).

Mais en fait, toutes les adresses font la même taille (sur la plupart des machines aujourd'hui, 8 octets), ce ne sont que des numéros d'octets dans la mémoire.

L'abandon des types

Le type `void *` est le type des pointeurs qui pointent vers des choses de type inconnu. Exemples bien connus :

```
void *malloc(size_t size);
void free(void *ptr);
void *memcpy(void *dest, const void *src, size_t n);
ssize_t write(int fd, const void *buf, size_t count);
ssize_t read(int fd, void *buf, size_t count);
```

- Utilisé lorsqu'on se fiche pas mal du type (par exemple, `memcpy` se contente d'écrire quelque part un paquet d'octets qui sont quelque part).
- Ou bien lorsqu'un type autre que `void *` serait trop restrictif. Par exemple, la fonction

  ```
  void qsort(void *base, size_t nmemb, size_t size,
                  int (*compar)(const void *, const void *));
  ```

  implémente le tri rapide pour n'importe quel type de données qui dispose d'une fonction de comparaison.

Ce qu'on peut faire avec `void *`

- Mettre n'importe quel type d'adresse dedans, sans *cast* nécessaire :

```
int n;
double x;
void *p;
p = &n;
p = &x;
```

- Mettre un `void *` dans n'importe quel type de pointeur, sans *cast* nécessaire :

```
int n;
void *p = &n;
int *m = p;
m = malloc(sizeof(int) * 4);
```

Ce qu'on ne peut pas faire avec `void *`

- On ne peut pas faire d'arithmétique avec (raison : on ignore le type pointé et donc sa taille). [1]

```
int tab[] = { 33, 78 };
void *p = tab;
++p; /* normalement interdit à la compilation */
```

- On ne peut pas déréférencer un pointeur de type `void *` (raison : on ignore le type pointé et donc le nombre d'octets à lire ou modifier).

```
void *p = malloc(12); /* un bloc de 12 octets */
*p = 45; /* interdit, erreur à la compilation */
int n = *p + 3; /* interdit, erreur à la compilation */
```

1. Une extension de `gcc`, active par défaut, permet en fait l'arithmétique sur `void *`, mais c'est interdit dans ce cours.

Motivation

Pointeurs génériques

Pointeurs de fonctions

Threads

Type pointeur de fonction

- `int img_de_0(int (*f) (int));`

  `img_de_0` a un paramètre (nommé `f`), de type *pointeur* de fonction prenant un paramètre `int` (type entre parenthèses) et retournant une valeur de type de `int`.
- `int compose(int (*f) (int), int (*g) (int), int n);`

  `compose` a deux arguments pointeurs vers fonctions prenant un `int` et retournant un `int` et un dernier argument `int`.

Fonction img_de_0 : définition

```
int img_de_0(int (*f) (int))
{
    /* on peut déréférencer le pointeur de fonction */
    return (*f)(0);
}
```

Fonction img_de_0 : définition

```
int img_de_0(int (*f) (int))
{
    /* on peut déréférencer le pointeur de fonction */
    return (*f)(0);
}
```

Facilité d'écriture : lorsqu'un pointeur de fonction est appliqué à des arguments, il est automatiquement déréférencé. On peut donc aussi écrire (en pratique, on préfère, c'est plus simple à écrire) :

```
int img_de_0(int (*f) (int))
{
    /* déréférencement implicite */
    return f(0);
}
```

Fonction img_de_0 : appel

```c
#include <stdio.h>

int img_de_0(int (*f) (int));
int triple(int n);
int plus_deux(int n);
int main(void)
{
    int n, m;
    /* adresse de triple donnée en argument. */
    n = img_de_0(&triple);
    /* en fait cast automatique en pointeur de fonction. */
    m = img_de_0(plus_deux);
    printf("n = %d, m = %d\n");
    return 0;
}
int img_de_0(int (*f) (int)) { return f(0); }
int triple(int n) { return n * 3; }
int plus_deux(int n) { return n + 2; }
```

Pointeurs de fonctions : récapitulatif

- Le type (pointeur de) fonction dont les paramètres ont les types `p1`, `p2`, ... `pn` et la valeur de retour est de type `r` est `r (*)(p1, p2,.., pn)`
- Partout où l'adresse d'une fonction (du bon type) est attendue, on peut donner le nom d'une telle fonction (sans opérateur `&`, ou avec, selon les goûts).
- Pour appeler une fonction à partir de son adresse `faddr`, on peut utiliser la syntaxe avec déréférencement `*faddr(arg1, arg2, ..., argn)` ou non `faddr(arg1, arg2, ..., argn)`.
- Dans ce cours, je vais au plus simple (pas d'opérateur `&` et pas d'opérateur `*`).

Pointeurs de fonction et pointeurs « simples »

Le standard du C n'autorise pas...

- les *casts* entre différents types de pointeurs de fonction
- les *casts* entre pointeurs de fonction et pointeurs « simples » sont interdits même avec `void *`
- Mais... `gcc` autorise dans certains cas certains *casts* entre types pointeurs de fonctions (et c'est utilisé dans la Glib).

Motivation

Pointeurs génériques

Pointeurs de fonctions

Threads

Hello, fork

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
void afficher(char *ch);
int main(void)
{
        pid_t p;
        char message[] = "Salut les L2 !\n";
        switch ((p = fork())) {
        case -1: perror("fork"); return 1;
        case 0: afficher(message);
        }
        wait(NULL);
        printf("Mon enfant a terminé.\n");
        printf("Le message était %s", message);
        return 0;
}
void afficher(char *ch)
{
        printf("%s\n", ch);
        exit(0);
}
```

Résumé

On a ici un programme qui crée deux processus :

- l'enfant affiche un message (contenu dans la pile de `main`)
- le parent attend la terminaison de l'enfant et affiche également ce message.

Hello, thread

```c
#include <stdio.h>
#include <pthread.h>
void *afficher(void *arg);
int main(void)
{
        pthread_t t;
        char message[] = "Salut les L2 !\n";

        if (pthread_create(&t, NULL, afficher, message) != 0) {
                perror("pthread_create");
                return 1;
        }
        pthread_join(t, NULL);

        printf("L'autre thread a terminé.\n");
        printf("Le message était %s", message);
        return 0;
}
void *afficher(void *arg)
{
        char *mess = arg;
        printf("%s", mess);
        return NULL;
}
```

Compilation et exécution

```
$ gcc -Wall hello_thread.c -pthread
$ ./a.out
Salut les L2 !
L'autre thread a terminé.
Le message était Salut les L2 !
```

Création d'un *thread*

```
pthread_t t;
pthread_create(&t, NULL, afficher, message);
```

- Un *thread* (en français *fil d'exécution* ou encore *processus léger*) est créé. C'est une nouvelle tâche, qui va être également ordonnancée par le système
- et donc possiblement exécutée en parallèle de la première tâche, si la machine et le contexte le permettent.
- La tâche commence son travail ici à la première instruction de `afficher`, qui *doit* impérativement :
  - avoir un seul paramètre, de type `void *` ;
  - retourner une valeur de type `void *`.
- lorsque cette fonction retourne, le *thread* est terminé ;
- le deuxième argument de `pthread_create` est `NULL` ici (attributs de *thread* par défaut) ;
- le dernier argument, de type `void *` est celui qui est transmis à la fonction `afficher` pour ce *thread*.

Identification d'un *thread*

- Le type opaque `pthread_t` permet de stocker un identifiant de *thread*[2] ;
- la fonction `pthread_create` met à l'adresse indiquée par son premier argument l'identifiant du *thread* créé ;
- de manière à pouvoir intéragir avec lui (par exemple attendre sa terminaison, l'annuler, ...).

2. chez moi, c'est `unsigned long`, mais ne pas faire d'hypothèse dessus pour écrire des programmes portables

Détection des erreurs

- En cas d'erreur, comme presque toutes les fonctions de la bibliothèque `pthread`, `pthread_create` retourne une valeur $\neq 0$ et remplit `errno` en conséquence.
- d'où l'appel dans `if` :

```
if (pthread_create(&t, NULL, afficher, message) != 0) {
    perror("pthread_create");
    return 1;
}
```

Attendre la terminaison d'un *thread* (1)

```
#include <pthread.h>
int pthread_join(pthread_t thread, void **retval);
```

- Dans l'exemple :

  `pthread_join(t, NULL);`
- Ici, le *thread* initial n'utilise pas la valeur de retour de la fonction `afficher` exécutée par l'autre *thread* donc le second argument est `NULL`
- Si on en a besoin, on doit mettre l'adresse d'une variable de type `void *` en second argument (voir exemple plus loin).

fork et *thread*

- *fork* $\leftrightarrow$ *create*
- *wait* $\leftrightarrow$ *join*
- Mais alors pourquoi s'embêter à apprendre les deux ?

Aucune différence ?

On change la fonction `afficher` dans les deux programmes :

```
/* dans hello_fork2.c */
void afficher(char *mess)
{
    mess[0] = 'H'; mess[1] = 'e'; mess[2] = 'l';
    mess[3] = 'l'; mess[4] = 'o';
    printf("%s", mess);
    exit(0);
}
```

et

```
/* dans hello_thread2.c */
void *afficher(void *arg)
{
    char *mess = arg;
    mess[0] = 'H'; mess[1] = 'e'; mess[2] = 'l';
    mess[3] = 'l'; mess[4] = 'o';
    printf("%s", mess);
    return NULL;
}
```

Aucune différence ?

```
$ gcc -Wall hello_fork2.c -o hello_fork2
$ ./hello_fork2
Hello les L2 !
Mon enfant a terminé.
Le message était Salut les L2 !
$ gcc -Wall hello_thread2.c -o hello_thread2 -pthread
$ ./hello_thread2
Hello les L2 !
L'autre thread a terminé.
Le message était Hello les L2 !
```

Avec des threads, la mémoire est partagée !

Exemple avec valeur de retour

```
void *f(void *arg);
int main(void)
{
        int tab[2] = { 33, 78 };
        pthread_t t;
        void *ret_addr;

        if (pthread_create(&t, NULL, f, tab) != 0) {
                perror("pthread_create");
                return 1;
        }
        pthread_join(t, &ret_addr);
        printf("%d + %d = %d\n", tab[0], tab[1], *((int *) ret_addr));
        free(ret_addr);
        return 0;
}

void *f(void *arg)
{
        int *tab = arg;
        int *resu = malloc(sizeof(int));
        *resu = tab[0] + tab[1];
        return resu;
}
```

Exemple avec valeur de retour

- le *thread* initial demande la création d'un second *thread* ;
- qui exécutera la fonction `f` ;
- avec l'adresse du tableau `tab` en argument.
- Le second *thread* commence par interpréter l'adresse transmise comme l'adresse d'un `int` ;

  ```
  int *tab = arg;
  ```

- puis alloue de la mémoire sur le tas pour stocker le résultat de son calcul [3] ;

  ```
  int *resu = malloc(sizeof(int));
  ```

- et retourne l'adresse de ce résultat.

3. en toute rigueur, on devrait tester si `malloc` a retourné `NULL`

Exemple avec valeur de retour

- Le *thread* initial attend la terminaison du second *thread*, puis met dans la variable `ret_addr` l'adresse transmise par celui-ci.

```
void *ret_addr;
pthread_join(t, &ret_addr);
```

- Il peut ensuite l'interpréter comme l'adresse d'un `int` et libérer la mémoire allouée. [4]

```
printf("%d + %d = %d\n", tab[0], tab[1], *((int *) ret_addr));
free(ret_addr);
```

4. En toute rigueur, il devrait vérifier que cette adresse n'est pas `NULL`.